

Reporte Técnico: Arquitectura de Sistemas de IA

Resumen Ejecutivo: Este documento demuestra la conversión fiel y elegante de Markdown a formato PDF, incluyendo formateo avanzado, tablas de métricas, código con sintaxis coloreada y fórmulas matemáticas.

1. Métricas de Rendimiento del Modelo

A continuación se presentan las comparativas de tiempo de inferencia y consumo de memoria:

Arquitectura	Precisión (%)	Latencia (ms)	Memoria VRAM	Estado
Transformer V4	98.4%	14.2 ms	4.2 GB	Producción
MoE Lite	96.8%	8.7 ms	2.1 GB	Beta
Dense Hybrid	99.1%	22.0 ms	6.8 GB	Evaluación

2. Modelado Matemático (KaTeX)

La función de optimización de pérdida ponderada se expresa formalmente como:

$$L(\theta) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] + \lambda \|\theta\|_2^2$$

Donde la tasa de aprendizaje adaptativa se rige por la ecuación de momento $v_t = \gamma v_{t-1} + \eta \nabla_{\theta} L(\theta)$.

3. Implementación en Código (Python & JavaScript)

Python: Pipeline de Inferencia

```
import numpy as np
from typing import List, Dict

class DocumentProcessor:
    """Procesador de alto rendimiento para extracción de texto."""

    def __init__(self, model_name: str = "deep-embed-v2"):
        self.model_name = model_name
        self.cache: Dict[str, np.ndarray] = {}

    def compute_embeddings(self, texts: List[str]) -> np.ndarray:
        print(f"Generando embeddings para {len(texts)} documentos...")
        embeddings = np.random.randn(len(texts), 768)
        return embeddings / np.linalg.norm(embeddings, axis=1, keepdims=True)

# Instancia del pipeline
processor = DocumentProcessor()
res = processor.compute_embeddings(["Reporte Q1", "Análisis de Datos"])
```

JavaScript: Consumo de la API de Conversión

```
async function convertDocToPdf(markdownContent) {
    const response = await fetch('http://localhost:3000/api/convert', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            markdown: markdownContent,
            theme: 'executive',
            format: 'A4'
        })
    });

    const blob = await response.blob();
    return URL.createObjectURL(blob);
}
```

4. Conclusiones y Próximos Pasos

- ☒ Optimizar la serialización de datos
- ☒ Integrar soporte nativo para diagramas y ecuaciones matemáticas
- ☐ Desplegar en clúster de producción
- ☐ Configurar monitoreo con telemetría en tiempo real

Generado automáticamente por MD-to-PDF Converter.